

5.7 专题：支持向量机

支持向量机（support vector machine, SVM）是非常强大、灵活的有监督学习算法，既可用于分类，也可用于回归。在本节中，我们将介绍支持向量机的原理，并用它解决分类问题。首先还是导入需要用的程序库：

```
In[1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# 用Seaborn画图
import seaborn as sns; sns.set()
```

5.7.1 支持向量机的由来

在前面介绍贝叶斯分类器（详情请参见 5.5 节）时，我们首先对每个类进行了随机分布的假设，然后用生成的模型估计新数据点的标签。那是生成分类方法，这里将介绍判别分类方法：不再为每类数据建模，而是用一条分割线（二维空间中的直线或曲线）或者流形体（多维空间中的曲线、曲面等概念的推广）将各种类型分割开。

下面用一个简单的分类示例来演示，其中两种类型的数据可以被清晰地分割开（如图 5-53 所示）：

```
In[2]: from sklearn.datasets.samples_generator import make_blobs
X, y = make_blobs(n_samples=50, centers=2,
                  random_state=0, cluster_std=0.60)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn');
```

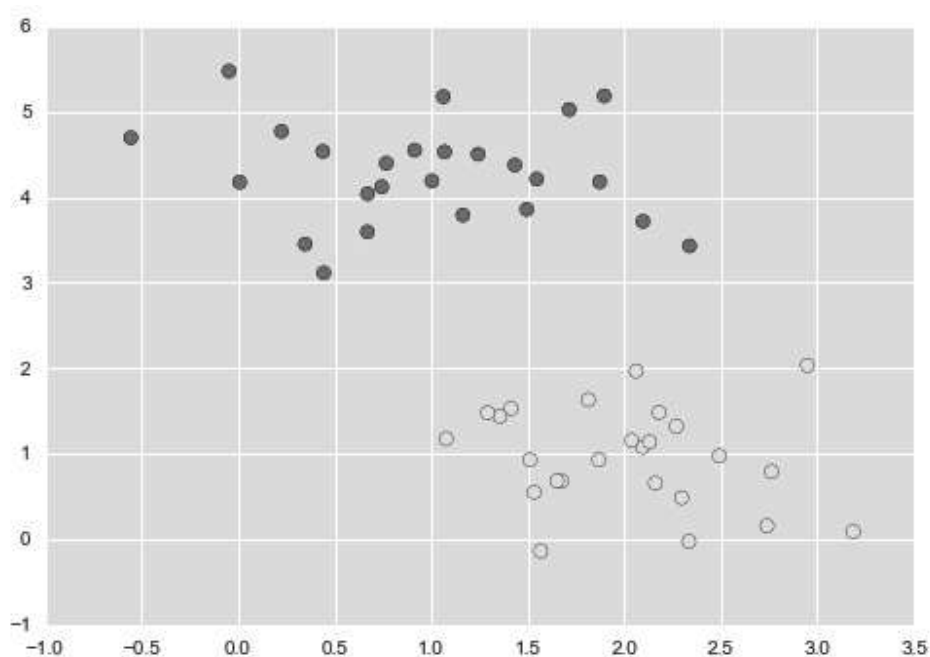


图 5-53: 简易分类数据

这个线性判别分类器尝试画一条将数据分成两部分的直线，这样就构成了一个分类模型。对于上图的二维数据来说，这个任务其实可以手动完成。但是我们马上发现一个问题：在这两种类型之间，有不止一条直线可以将它们完美分割。

可以把它们画出来看看（如图 5-54 所示）：

```
In[3]: xfit = np.linspace(-1, 3.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plt.plot([0.6], [2.1], 'x', color='red', markeredgewidth=2, markersize=100)

for m, b in [(1, 0.65), (0.5, 1.6), (-0.2, 2.9)]:
    plt.plot(xfit, m * xfit + b, '-k')

plt.xlim(-1, 3.5);
```

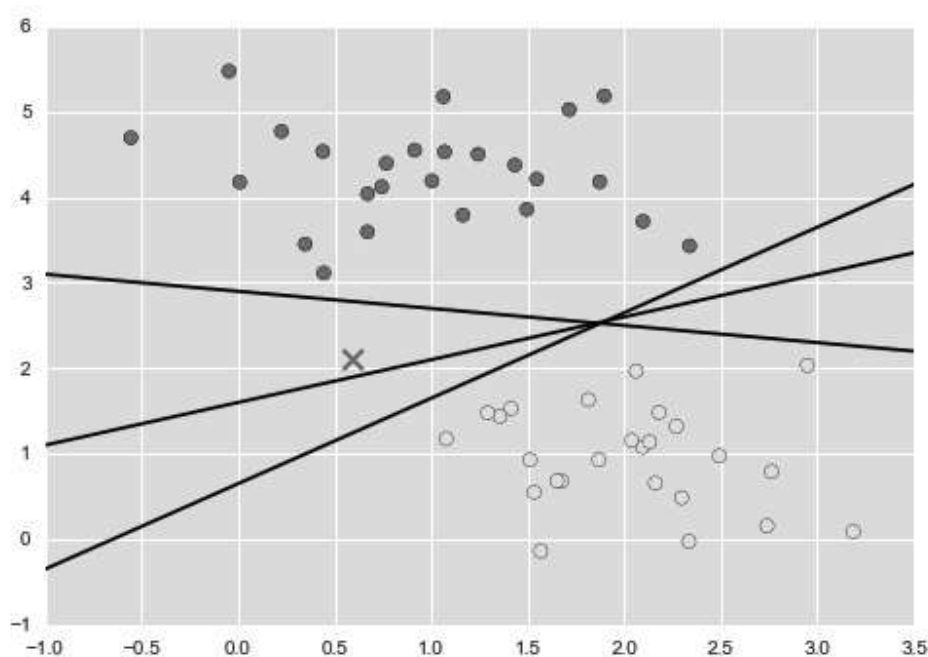


图 5-54：三条完美的线性判别分类器

虽然这三个不同的分割器都能完美地判别这些样本，但是选择不同的分割线，可能会让新的数据点（例如图 5-54 中的“X”点）分配到不同的标签。显然，“画一条分割不同类型的直线”还不够，我们需要进一步思考。

5.7.2 支持向量机：边界最大化

支持向量机提供了改进这个问题的方法，它直观的解释是：不再画一条细线来区分类型，而是画一条到最近点边界、有宽度的线条。具体形式如下面的示例所示（如图 5-55 所示）：

```
In[4]:
xfit = np.linspace(-1, 3.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')

for m, b, d in [(1, 0.65, 0.33), (0.5, 1.6, 0.55), (-0.2, 2.9, 0.2)]:
    yfit = m * xfit + b
    plt.plot(xfit, yfit, '-k')
    plt.fill_between(xfit, yfit - d, yfit + d, edgecolor='none', color='#AA
                    alpha=0.4)
```

```
plt.xlim(-1, 3.5);
```

在支持向量机中，选择边界最大的那条线是模型最优解。支持向量机其实就是一个边界最大化评估器。

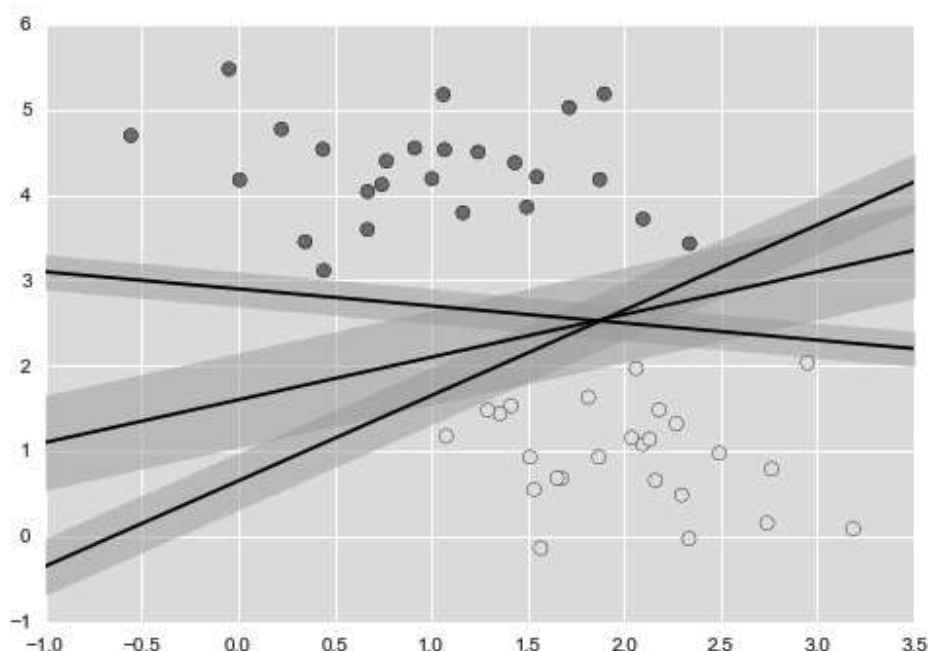


图 5-55：带“边界”的判别分类器

01. 拟合支持向量机

来看看这个数据的真实拟合结果：用 Scikit-Learn 的支持向量机分类器在数据上训练一个 SVM 模型。这里用一个线性核函数，并将参数 **C** 设置为一个很大的数（后面会介绍这些设置的意义）：

```
In[5]: from sklearn.svm import SVC # "Support vector classifier"
       model = SVC(kernel='linear', C=1E10)
       model.fit(X, y)

Out[5]: SVC(C=10000000000.0, cache_size=200, class_weight=None, coef0=
        decision_function_shape=None, degree=3, gamma='auto', kernel=
        max_iter=-1, probability=False, random_state=None, shrinking=
        tol=0.001, verbose=False)
```

为了实现更好的可视化分类效果，创建一个辅助函数画出 SVM 的决策边界（如图 5-56 所示）：

```
In[6]: def plot_svc_decision_function(model, ax=None, plot_support=True):
        """画二维SVC的决策函数"""
        if ax is None:
            ax = plt.gca()
        xlim = ax.get_xlim()
        ylim = ax.get_ylim()

        # 创建评估模型的网格
        x = np.linspace(xlim[0], xlim[1], 30)
        y = np.linspace(ylim[0], ylim[1], 30)
        Y, X = np.meshgrid(y, x)
        xy = np.vstack([X.ravel(), Y.ravel()]).T
        P = model.decision_function(xy).reshape(X.shape)

        # 画决策边界和边界
        ax.contour(X, Y, P, colors='k',
                   levels=[-1, 0, 1], alpha=0.5,
                   linestyles=['--', '-', '--'])

        # 画支持向量
        if plot_support:
            ax.scatter(model.support_vectors_[0],
                       model.support_vectors_[1],
                       s=300, linewidth=1, facecolors='none');
        ax.set_xlim(xlim)
        ax.set_ylim(ylim)

In[7]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
        plot_svc_decision_function(model);
```

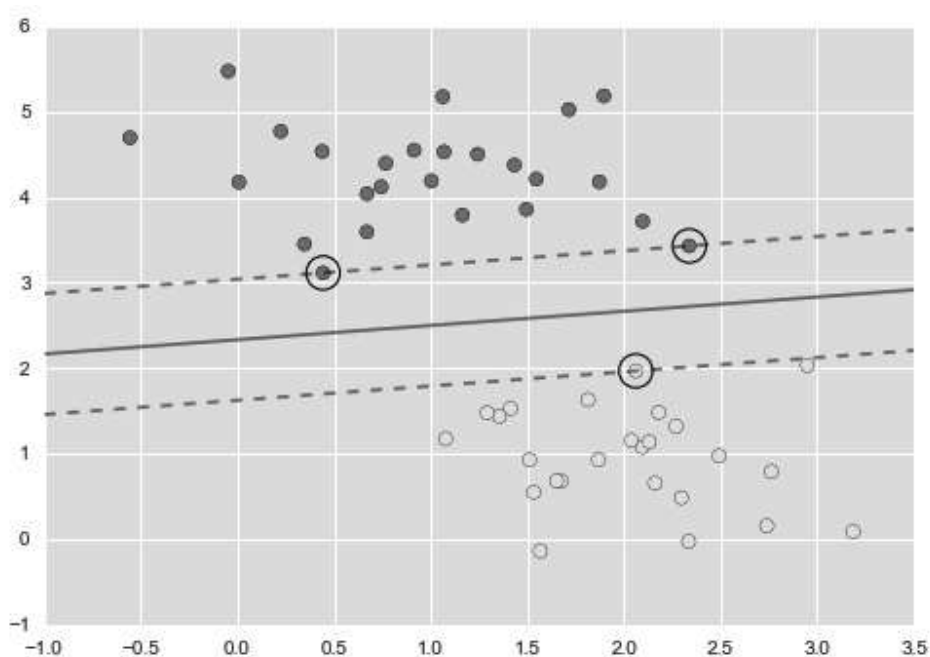


图 5-56: 带边界线（虚线）和支持向量（圆圈）的支持向量机分类器拟合数据结果

这就是两类数据间隔最大的分割线。你会发现有一些点正好就在边界线上，在图 5-56 中用黑圆圈表示。这些点是拟合的关键支持点，被称为支持向量，支持向量机算法也因此得名。在 Scikit-Learn 里面，支持向量的坐标存放在分类器的 `support_vectors_` 属性中：

```
In[8]: model.support_vectors_  
  
Out[8]: array([[ 0.44359863,  3.11530945],  
               [ 2.33812285,  3.43116792],  
               [ 2.06156753,  1.96918596]])
```

分类器能够成功拟合的关键因素，就是这些支持向量的位置——任何在正确分类一侧远离边界线的点都不会影响拟合结果！从技术角度解释的话，是因为这些点不会对拟合模型的损失函数产生任何影响，所以只要它们没有跨越边界线，它们的位置和数量就都无关紧要。

例如，可以分别画出数据集前 60 个点和前 120 个点的拟合结果，

并进行对比（如图 5-57 所示）：

```
In[9]: def plot_svm(N=10, ax=None):
        X, y = make_blobs(n_samples=N, centers=2,
                           random_state=0, cluster_std=0.60)

        X = X[:N]
        y = y[:N]
        model = SVC(kernel='linear', C=1E10)
        model.fit(X, y)

        ax = ax or plt.gca()
        ax.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
        ax.set_xlim(-1, 4)
        ax.set_ylim(-1, 6)
        plot_svc_decision_function(model, ax)

fig, ax = plt.subplots(1, 2, figsize=(16, 6))
fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)
for axi, N in zip(ax, [60, 120]):
    plot_svm(N, axi)
    axi.set_title('N = {}'.format(N))
```

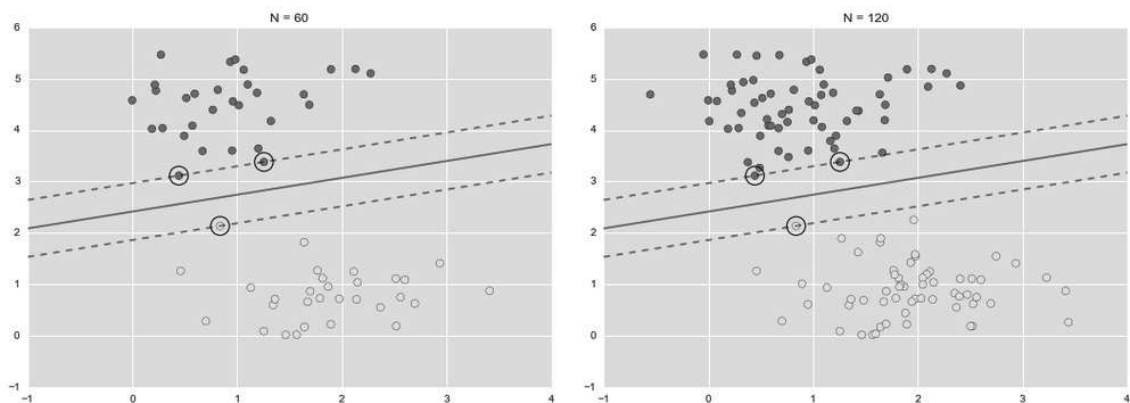


图 5-57：新训练数据点对 SVM 模型的影响

我们在左图中看到的是前 60 个训练样本的模型和支持向量。在右图中，虽然我们画了前 120 个训练样本的支持向量，但是模型并没有改变：左图中的 3 个支持向量仍然适用于右图。这种对远离边界的数据点不敏感的特点正是 SVM 模型的优点之一。

如果你正在运行 Notebook，可以用 IPython 的交互组件动态观察 SVM 模型的这个特点（如图 5-58 所示）：

```
In[10]: from ipywidgets import interact, fixed
        interact(plot_svm, N=[10, 200], ax=fixed(None));
```

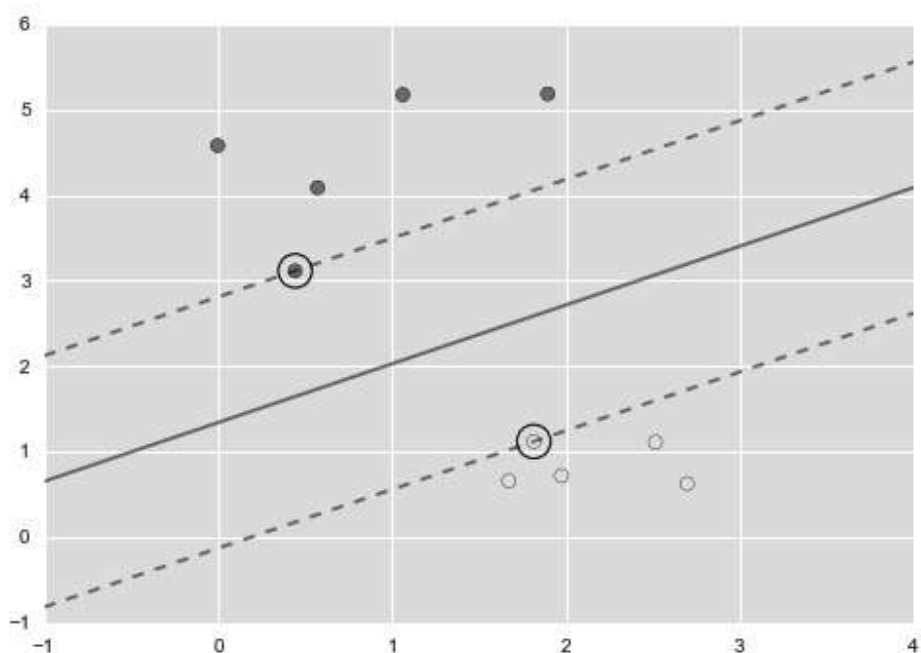


图 5-58: SVM 模型可视化的第一帧画面（详情请参考 **Github** 上的完整在线附录，<https://github.com/jakevdp/PythonDataScienceHandbook>）

02. 超越线性边界：核函数SVM模型

将 SVM 模型与核函数组合使用，功能会非常强大。我们在 5.6 节介绍基函数回归时介绍过一些核函数。那时，我们将数据投影到多项式和高斯基函数定义的高维空间中，从而实现用线性分类器拟合非线性关系。

在 SVM 模型中，我们可以沿用同样的思路。为了应用核函数，引入一些非线性可分的数据（如图 5-59 所示）：

```
In[11]: from sklearn.datasets.samples_generator import make_circles
        X, y = make_circles(100, factor=.1, noise=.1)

        clf = SVC(kernel='linear').fit(X, y)
```



```
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(clf, plot_support=False);
```

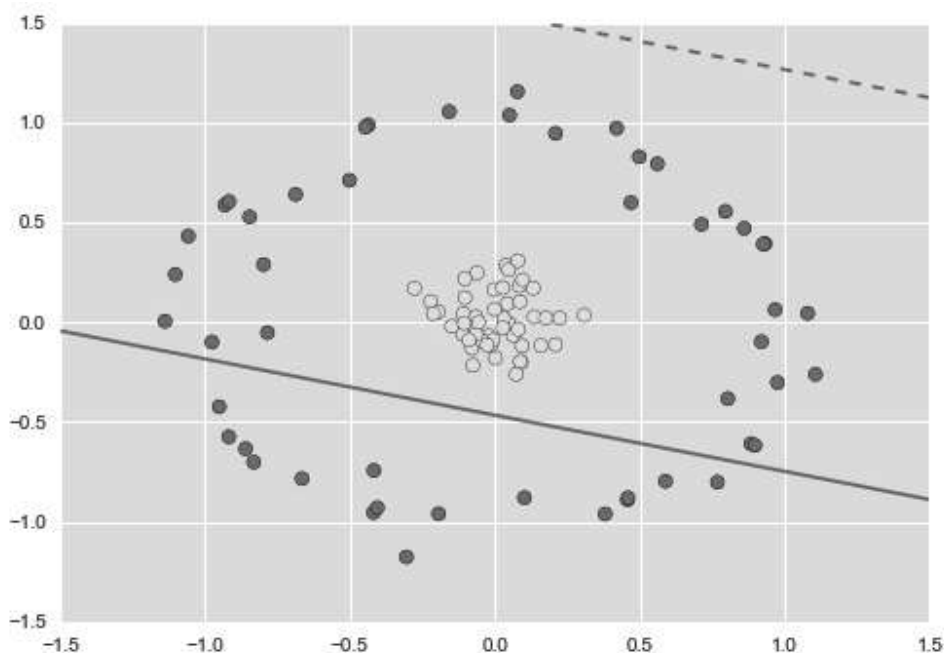


图 5-59：用线性分类器处理非线性边界的效果

显然，这里需要用非线性判别方法来分割数据。回顾一下 5.6 节介绍的基函数回归方法，想想如何将数据投影到高维空间，从而使线性分割器可以派上用场。例如，一种简单的投影方法就是计算一个以数据圆圈（middle clump）为中心的径向基函数：

```
In[12]: r = np.exp(-(X ** 2).sum(1))
```

可以通过三维图来可视化新增的维度——如果你正在运行 Notebook，就可以用滑块变换观察角度（如图 5-60 所示）：

```
In[13]: from mpl_toolkits import mplot3d

def plot_3D(elev=30, azim=30, X=X, y=y):
    ax = plt.subplot(projection='3d')
    ax.scatter3D(X[:, 0], X[:, 1], r, c=y, s=50, cmap='autumn')
    ax.view_init(elev=elev, azim=azim)
    ax.set_xlabel('x')
```

```
ax.set_ylabel('y')
ax.set_zlabel('r')

interact(plot_3D, elev=[-90, 90], azip=(-180, 180),
        X=fixed(X), y=fixed(y));
```

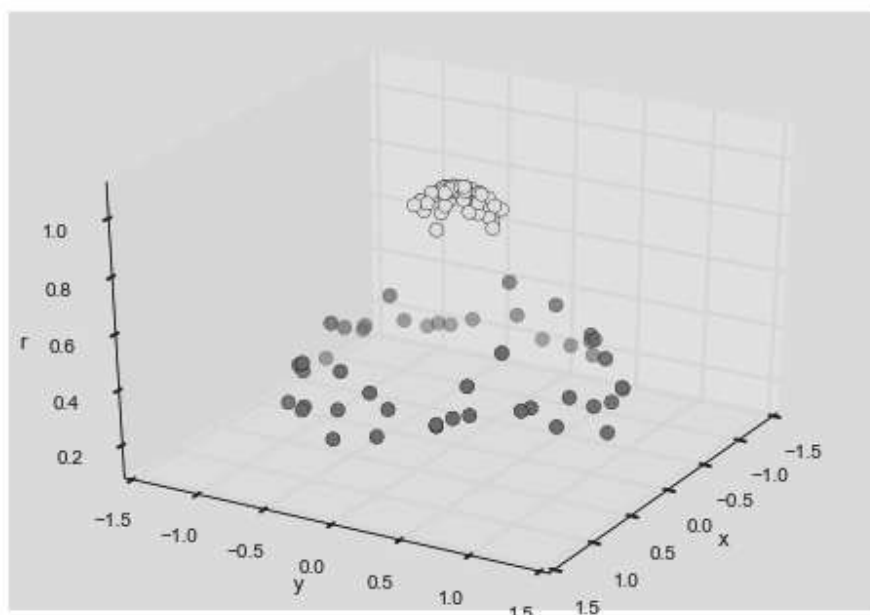


图 5-60： 可以进行线性分割的第三个维度

增加新维度之后，数据变成了线性可分状态。如果现在画一个分割平面，例如 $r = 0.7$ ，即可将数据分割。

我们还需要仔细选择和优化投影方式；如果不能将径向基函数集中到正确的位置，那么就得不到如此干净、可分割的结果。通常，选择基函数比较困难，我们需要让模型自动指出最合适的基函数。

一种策略是计算基函数在数据集上每个点的变换结果，让 SVM 算法从所有结果中筛选出最优解。这种基函数变换方式被称为核变换，是基于每对数据点之间的相似度（或者核函数）计算的。

这种策略的问题是，如果将 N 个数据点投影到 N 维空间，当 N 不断增大的时候就会出现维度灾难，计算量巨大。但由于核函数技巧（<http://bit.ly/2fStZeA>）提供的小程序可以隐式计算核变换数据的拟合，也就是说，不需要建立完全的 N 维核函数投影空间！这个核函

数技巧内置在 SVM 模型中，是使 SVM 方法如此强大的充分条件。

在 Scikit-Learn 里面，我们可以应用核函数化的 SVM 模型将线性核转变为 RBF（径向基函数）核，设置 `kernel` 模型超参数即可（如图 5-61 所示）：

```
In[14]: clf = SVC(kernel='rbf', C=1E6)
        clf.fit(X, y)

Out[14]: SVC(C=1000000.0, cache_size=200, class_weight=None, coef0=0.0,
            decision_function_shape=None, degree=3, gamma='auto', kernel='rbf',
            max_iter=-1, probability=False, random_state=None, shrinking=True,
            tol=0.001, verbose=False)

In[15]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
        plot_svc_decision_function(clf)
        plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
                    s=300, lw=1, facecolors='none');
```

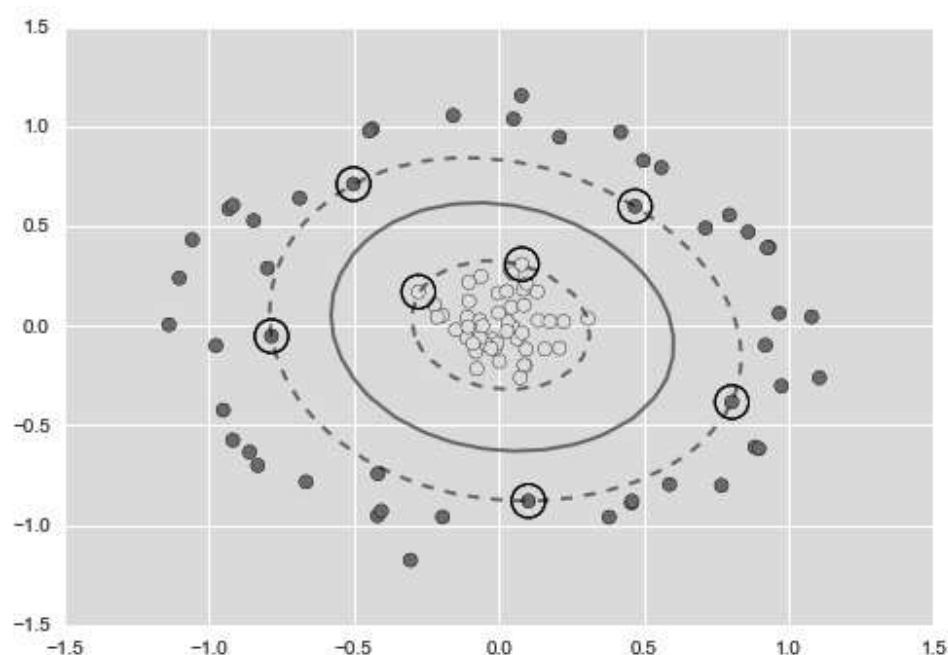


图 5-61：核函数化的 SVM 模型拟合数据

通过使用这个核函数化的支持向量机，我们找到了一条合适的非线

性决策边界。在机器学习中，核变换策略经常用于将快速线性方法变换成快速非线性方法，尤其是对于那些可以应用核函数技巧的模型。

03. SVM优化：软化边界

到目前为止，我们介绍的模型都是在处理非常干净的数据集，里面都有非常完美的决策边界。但如果你的数据有一些重叠该怎么办呢？例如，有如下所示一些数据（如图 5-62 所示）：

```
In[16]: X, y = make_blobs(n_samples=100, centers=2,
                           random_state=0, cluster_std=1.2)
        plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn');
```

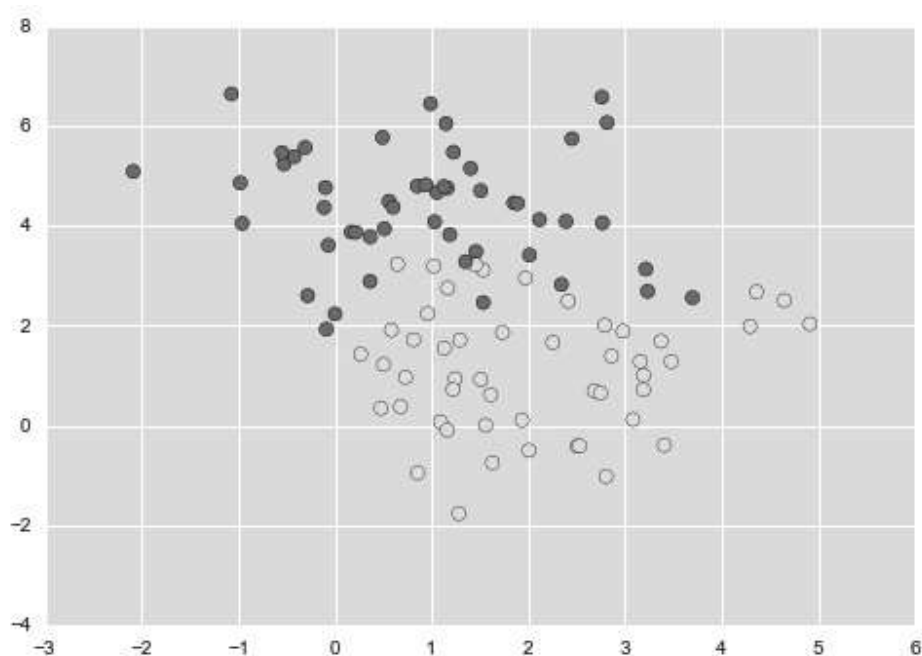


图 5-62：有重叠的数据

为了解决这个问题，SVM 实现了一些修正因子来“软化”边界。为了取得更好的拟合效果，它允许一些点位于边界线之内。边界线的硬度可以通过超参数进行控制，通常是 C 。如果 C 很大，边界就会很硬，数据点便不能在边界内“生存”；如果 C 比较小，边界线比较软，有一些数据点就可以穿越边界线。

图 5-63 显示了不同参数 C 通过软化边界线，对拟合效果产生的影响：

```
In[17]: X, y = make_blobs(n_samples=100, centers=2,
                           random_state=0, cluster_std=0.8)

fig, ax = plt.subplots(1, 2, figsize=(16, 6))
fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)

for axi, C in zip(ax, [10.0, 0.1]):
    model = SVC(kernel='linear', C=C).fit(X, y)
    axi.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
    plot_svc_decision_function(model, axi)
    axi.scatter(model.support_vectors_[:, 0],
                model.support_vectors_[:, 1],
                s=300, lw=1, facecolors='none');
    axi.set_title('C = {0:.1f}'.format(C), size=14)
```

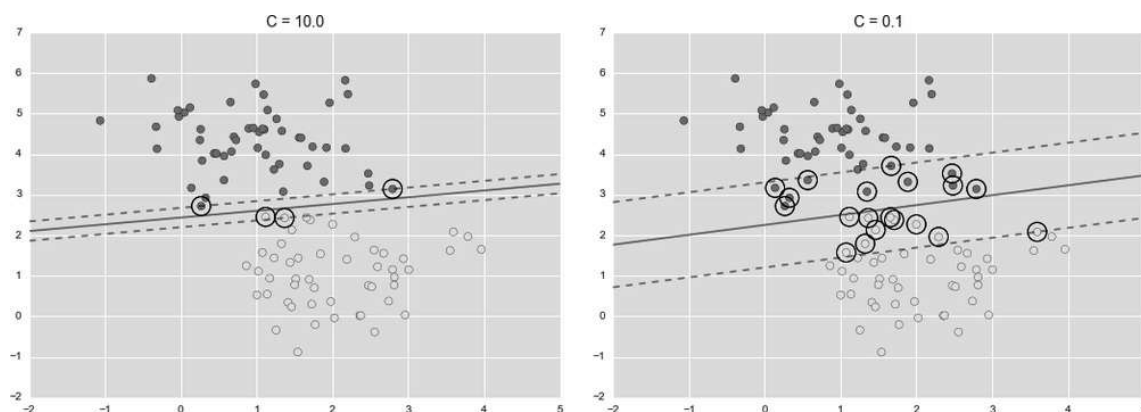


图 5-63：不同参数 C 的支持向量机拟合效果

参数 C 的最优值视数据集的具体情况而定，通过交叉检验或者类似的程序进行计算（详情请参见 5.3 节）。

5.7.3 案例：人脸识别

我们用人脸识别案例来演示支持向量机的实战过程。这里用 Wild 数据集中带标记的人脸图像，里面包含了数千张公开的人脸照片。Scikit-Learn 内置了获取照片数据集的功能：

```
In[18]: from sklearn.datasets import fetch_lfw_people
        faces = fetch_lfw_people(min_faces_per_person=60)
        print(faces.target_names)
        print(faces.images.shape)

['Ariel Sharon' 'Colin Powell' 'Donald Rumsfeld' 'George W Bush'
 'Gerhard Schroeder' 'Hugo Chavez' 'Junichiro Koizumi' 'Tony Blair']
(1348, 62, 47)
```

画一些人脸，看看需要处理的数据（如图 5-64 所示）：

```
In[19]: fig, ax = plt.subplots(3, 5)
        for i, axi in enumerate(ax.flat):
            axi.imshow(faces.images[i], cmap='bone')
            axi.set(xticks=[], yticks=[],
                    xlabel=faces.target_names[faces.target[i]])
```

每个图像包含 [62×47]、接近 3000 像素。虽然可以简单地将每个像素作为一个特征，但是更高效的方法通常是使用预处理器来提取更有意义的特征。这里使用主成分分析（详情请参见 5.9 节）来提取 150 个基本元素，然后将其提供给支持向量机分类器。可以将这个预处理器和分类器打包成管道：

```
In[20]: from sklearn.svm import SVC
        from sklearn.decomposition import RandomizedPCA
        from sklearn.pipeline import make_pipeline

        pca = RandomizedPCA(n_components=150, whiten=True, random_state=42)
        svc = SVC(kernel='rbf', class_weight='balanced')
        model = make_pipeline(pca, svc)
```

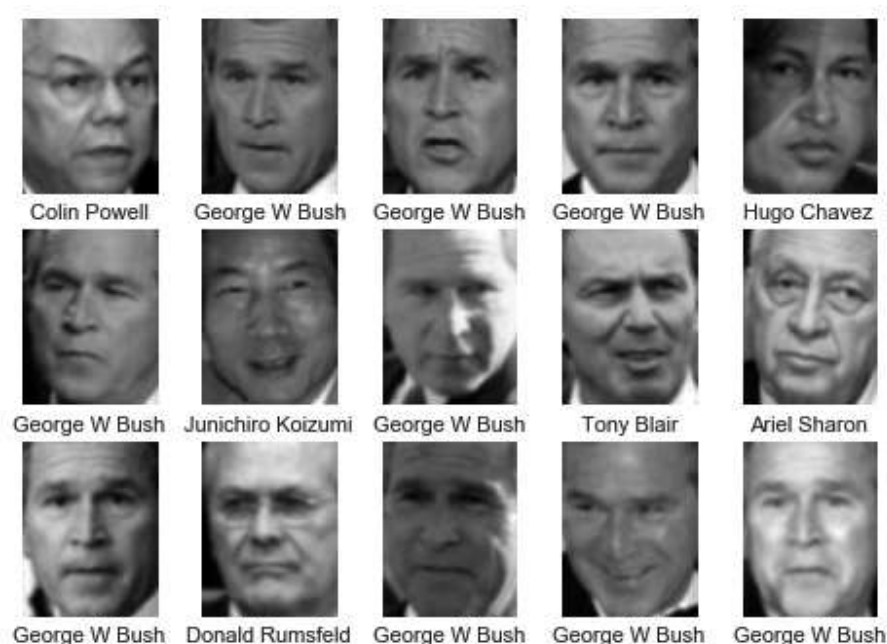


图 5-64: **Wild** 数据集中带标记的人脸图像

为了测试分类器的训练效果，将数据集分解成训练集和测试集进行交叉检验：

```
In[21]: from sklearn.cross_validation import train_test_split
        Xtrain, Xtest, ytrain, ytest = train_test_split(faces.data, faces.t
                                                    random_state=42)
```

最后，用网格搜索交叉检验来寻找最优参数组合。通过不断调整参数 **C**（控制边界线的硬度）和参数 **gamma**（控制径向基函数核的大小），确定最优模型：

```
In[22]: from sklearn.grid_search import GridSearchCV
        param_grid = {'svc__C': [1, 5, 10, 50],
                      'svc__gamma': [0.0001, 0.0005, 0.001, 0.005]}
        grid = GridSearchCV(model, param_grid)

        %time grid.fit(Xtrain, ytrain)
        print(grid.best_params_)

CPU times: user 47.8 s, sys: 4.08 s, total: 51.8 s
Wall time: 26 s
{'svc__gamma': 0.001, 'svc__C': 10}
```

最优参数最终落在了网格的中间位置。如果它们落在了边缘位置，我们可能就需要扩展网格搜索范围，确保最优参数可以被搜索到。

有了交叉检验的模型，现在就可以对测试集的数据进行预测了：

```
In[23]: model = grid.best_estimator_  
        yfit = model.predict(Xtest)
```

将一些测试图片与预测图片进行对比（如图 5-65 所示）：

```
In[24]: fig, ax = plt.subplots(4, 6)  
        for i, axi in enumerate(ax.flat):  
            axi.imshow(Xtest[i].reshape(62, 47), cmap='bone')  
            axi.set(xticks=[], yticks=[])  
            axi.set_ylabel(faces.target_names[yfit[i]].split()[-1],  
                        color='black' if yfit[i] == ytest[i] else 'red')  
        fig.suptitle('Predicted Names; Incorrect Labels in Red', size=14);
```

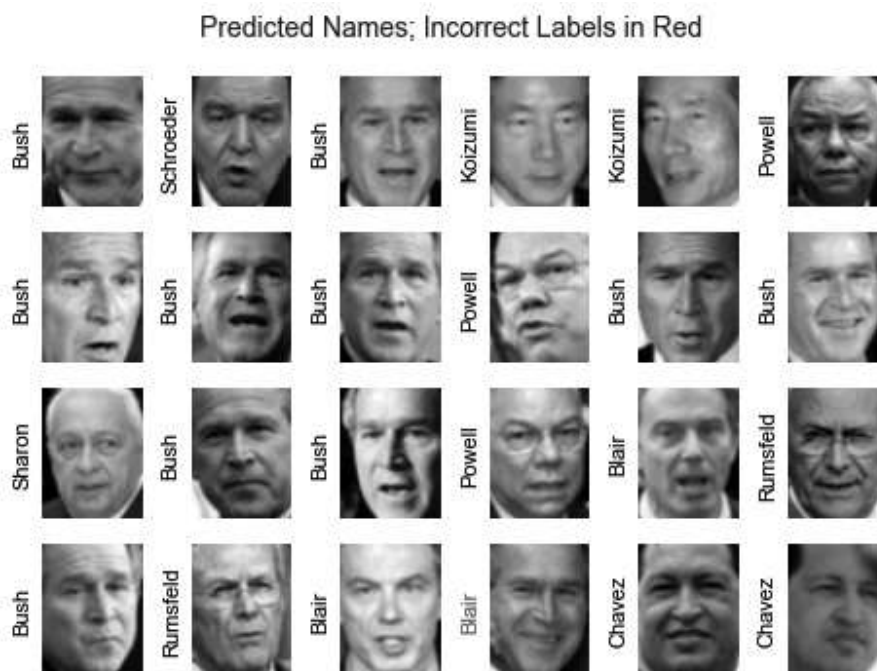


图 5-65：模型预测标签

在这个小样本中，我们的最优评估器只判断错了一张照片（最后一行中布什的照片被误判为布莱尔）。我们可以打印分类效果报告，它会列举每个标签的统计结果，从而对评估器的性能有更全面的认识：

```
In[25]: from sklearn.metrics import classification_report
        print(classification_report(ytest, yfit,
                                    target_names=faces.target_names))
```

	precision	recall	f1-score	support
Ariel Sharon	0.65	0.73	0.69	15
Colin Powell	0.81	0.87	0.84	68
Donald Rumsfeld	0.75	0.87	0.81	31
George W Bush	0.93	0.83	0.88	126
Gerhard Schroeder	0.86	0.78	0.82	23
Hugo Chavez	0.93	0.70	0.80	20
Junichiro Koizumi	0.80	1.00	0.89	12
Tony Blair	0.83	0.93	0.88	42
avg / total	0.85	0.85	0.85	337

还可以画出这些标签的混淆矩阵（如图 5-66 所示）：

```
In[26]: from sklearn.metrics import confusion_matrix
        mat = confusion_matrix(ytest, yfit)
        sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False,
                    xticklabels=faces.target_names,
                    yticklabels=faces.target_names)
        plt.xlabel('true label')
        plt.ylabel('predicted label');
```

	Ariel Sharon	11	2	1	2	0	1	0	0
	Colin Powell	1	59	2	11	0	0	0	0
	Donald Rumsfeld	2	2	27	3	1	0	0	1
predicted label	George W Bush	1	3	0	105	1	2	0	1
	Gerhard Schroeder	0	0	0	1	18	2	0	0
	Hugo Chavez	0	0	0	1	0	14	0	0
	Junichiro Koizumi	0	0	0	1	1	0	12	1
	Tony Blair	0	2	1	2	2	1	0	39
		Ariel Sharon	Colin Powell	Donald Rumsfeld	George W Bush	Gerhard Schroeder	Hugo Chavez	Junichiro Koizumi	Tony Blair
		true label							

图 5-66：人脸数据混淆矩阵

这幅图可以帮助我们清晰地判断哪些标签容易被分类器误判。

真实环境中的人脸识别问题的照片通常不会被切割得这么整齐（即使像素相同），两类人脸分类机制的唯一差别其实是特征选择：你需要用更复杂的算法找到人脸，然后提取图片中与像素化无关的人脸特征。这类问题有一个不错的解决方案，就是用 OpenCV (<http://opencv.org>) 配合其他手段，包括最先进的通用图像和人脸图像的特征提取工具，来获取人脸特征数据。

5.7.4 支持向量机总结

前面已经简单介绍了支持向量机的基本原则。支持向量机是一种强大的分类方法，主要有四点理由。

- 模型依赖的支持向量比较少，说明它们都是非常精致的模型，消耗内存少。